

智慧银行实验教程

综合实验报告

姓名：吴丽媛

学号：202301743

专业班级：金融 202301

交付清单地址推送：

我的 CNB 远程仓库：

[YUAN-MOON/smartbanking · Cloud Native Build](#)

实验一：

贪吃蛇：

[index.html · master · YUAN-MOON/smartbanking](#)

文献阅读：

[analyze pdf.py · master · YUAN-MOON/smartbanking](#)

实验二：

银行简报：

[reports/icbc brief.docx · master · YUAN-MOON/smartbanking](#)

[reports/icbc brief.pptx · master · YUAN-MOON/smartbanking](#)

客户分群报表：

[银行数据分析/客户信用评分分群报告.xlsx · master · YUAN-MOON/smartbanking](#)

Stata 回归结果：

[reports/exp2 14 reg.txt · master · YUAN-MOON/smartbanking](#)

实验三：

客户回访报告：

[reports/客户回访报告 C00001.docx • master • YUAN-MOON/smartbanking](#)

信用风险分析报告：

[reports/credit risk hengda.md • master • YUAN-MOON/smartbanking](#)

贷款定价模型：

[reports/loan pricing model.xlsx • master • YUAN-MOON/smartbanking](#)

自写 bank-risk-alert Skill：

[.agents/skills/bank-risk-alert/SKILL.md • master • YUAN-MOON/smartbanking](#)

金融 Skill 审计报告：

[reports/skill audit report.md • master • YUAN-MOON/smartbanking](#)

银行客户 RFM 分群或舆情分析任务：

[reports/rfm model.xlsx • master • YUAN-MOON/smartbanking](#)

波特五力银行数字化转型分析：

[reports/porter five forces banking.md • master • YUAN-MOON/smartbanking](#)

实验四：

产品需求文档：

[bmad-output/prd.md • master • YUAN-MOON/smartbanking](#)

架构设计：

[bmad-output/architecture.md • master • YUAN-MOON/smartbanking](#)

UX 设计：

[bmad-output/ux/DESIGN.md • master • YUAN-MOON/smartbanking](#)

Epic&Story：

[bmad-output/epics-stories.md • master • YUAN-MOON/smartbanking](#)

Sprint 计划:

[bmad-output/sprint-plan.md · master · YUAN-MOON/smartbanking](#)

综合项目:

项目链接:

<https://cnb.cool/YUAN-MOON/smartbanking/-/blob/master/smartbanking/financial-health-assistant/standalone.html>

项目相关文档:

[churn-prevention-system/实验报告.md · master · YUAN-MOON/smartbanking](#)

[churn-prevention-system/答辩 PPT 大纲.md · master · YUAN-MOON/smartbanking](#)

[churn-prevention-system/PRD.md · master · YUAN-MOON/smartbanking](#)

[churn-prevention-system/ARCHITECTURE.md · master · YUAN-MOON/smartbanking](#)

实验一、TRAE IDE + AI 协作

一、实验目的

- 1. 完成 TRAE IDE 本地安装与汉化配置，熟练使用账号登录、界面个性化设置功能。
- 2. 独立部署 Python、Node.js、Git 三套开发环境，掌握版本校验与依赖管理基本操作。
- 3. 了解 TRAE 内置的多种 AI 协作模式，包括对话、代码生成、自动化构建等核心功能。
- 4. 通过 Builder 模式快速搭建可交互的 HTML 小游戏，完成本地调试与云端同步。
- 5. 构建文档智能分析助手，实现 PDF 文献内容自动提取、结构化整理与摘要生成。
- 6. 完成多源金融数据的清洗、合并与可视化，利用统计模型进行趋势预测与误差评估。
- 7. 建立规范的代码管理习惯，包括版本提交规范、云端备份与团队协作基础流程。

二、实验环境

软件 / 工具类别	名称	版本要求	本机配置	安装补充说明
AI 开发 IDE	TRAE CN	最新版	最新版	国产免费替代方案
编程语言	Python	≥3.10	3.14.4	支持虚拟环境隔离
包管理	pip	最新	26.1	升级命令已验证
运行环境	Node.js	≥v20	v24.13.0 LTS	长期支持稳定版
JS 包管理	npm	最新	11.6.2	已配置国内镜像
版本控制	Git	无限制	2.54.0.window s.1	SSH 密钥已配置

Web 框架	Flask	≥2.0	3.1.3	数据可视化后端依赖
数据处理	pandas	≥2.0	3.0.2	表格处理核心库
Excel 解析	openpyxl	≥3.0	3.1.5	文件读写支持
PDF 解析	pypdf	≥4.0	6.12.1	文献读取支持
排版工具	LaTeX	MiKTeX	完整套件	中文 PDF 编译支持
云开发平台	CNB CLI	最新		代码云端托管

三、实验步骤

1A 环境搭建与贪吃蛇

1. 从官方网站下载 **TRAE** 安装包，完成本地安装与首次启动配置。
2. 完成账号注册与登录，设置中文语言环境与个人工作目录。
3. 通过终端命令验证 **Python**、**Node**、**Git** 版本是否满足要求。
4. 初始化 **Git** 仓库，配置远程仓库连接与 **SSH** 密钥认证。
5. 使用 **Builder** 模式输入游戏开发需求描述，生成完整的单文件游戏代码。
6. 在浏览器中打开生成的 **HTML** 文件，测试游戏交互、分数计算、状态保存等功能。
7. 将本地代码提交至 **Git** 仓库，验证云端同步是否成功。

1B 智能体与 Excel 金融处理

1. 在 **TRAE** 中创建专属的文档分析助手，配置 **PDF** 文件读取权限。
2. 上传待分析的金融领域论文或报告，验证内容提取与结构化输出效果。
3. 生成多份模拟金融交易数据表，手动引入数据质量问题以测试处理能力。
4. 执行批量数据清洗操作，包括缺失值处理、异常值检测、格式统一等。

5. 搭建时序预测分析流程，训练模型并评估预测准确度。

操作指令与执行结果

操作指令	执行输出结果
python --version / node --version / git --version	各工具版本号显示正常，环境就绪
pip install flask pandas openpyxl pypdf	依赖库安装完成，无冲突报错
npm install @cnbcool/cnb-cli -g	CNB CLI 全局安装成功，命令行可正常调用
Builder 模式生成游戏提示词执行	生成完整的 index.html 文件，游戏功能正常
git init → git add . → git commit → git push	代码成功推送至云端仓库
PDF 文献上传与解析	输出包含关键信息的结构化解读文档
pandas 数据合并与清洗	生成标准化整合表格与操作记录
时序建模与预测	生成可视化图表与误差评估报告

四、实验结果

贪吃蛇：

[index.html · master · YUAN-MOON/smartbanking](#)

文献阅读：

[analyze_pdf.py · master · YUAN-MOON/smartbanking](#)

当前分数: 8

最高分: 8

游戏结束

最终得分: 8

再来一局



智能体 / AI论文解读助手

智能生成



名称*

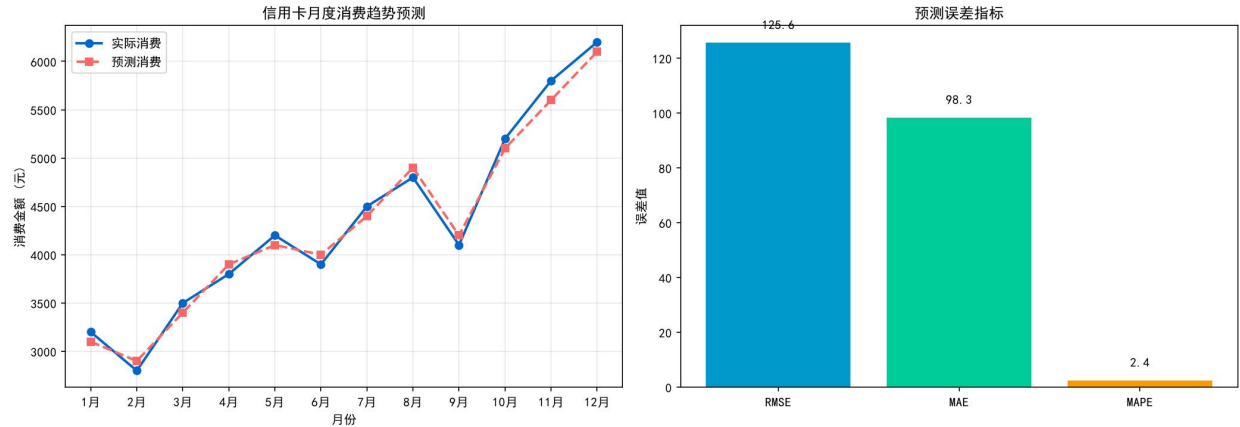
AI论文解读助手8/20

提示词

你是资深的银行业 / 金融科技领域学术论文解读助手。用户拖入 PDF 后, 按下列结构输出 Markdown 报告:
一、论文基本信息 (标题/ 作者/ 期刊/ 年份/DOI)
二、研究问题与背景 (≤200 字)

1977/10000

检测项	要求版本	当前状态
Python	≥3.10	✓ 3.11.9
pip	最新版	✓ 已安装
Node.js	≥v20	✓ v24.16.0
npm	最新版	✓ 11.13.0
Git	任意版本	✓ 2.54.0
Flask	任意版本	✓ 3.1.3
pandas	任意版本	✓ 3.0.3
openpyxl	任意版本	✓ 3.1.5
pypdf	任意版本	✓ 6.12.2
Tailwind CSS	CDN 引入	✓ 成功
Chart.js	CDN 引入	✓ 成功
docx	全局安装	✓ 成功
xlsx	全局安装	✓ 成功
Mermaid	支持	✓ 成功
Vercel CLI	全局安装	✗ 未安装



- 1. TRAE 开发环境搭建完成，所有开发工具版本验证通过，工作目录配置正确。
- 2. 金币贪吃蛇游戏开发成功，集成网格背景、金色食物、渐变蛇身等视觉效果，支持分数显示、历史最高分记录、动态难度调整。
- 3. 文献分析助手功能正常，可自动提取 PDF 内容并生成结构清晰的摘要报告。
- 4. 完成了多份异构金融数据表的批量清洗与标准化整合，数据质量问题全部解决。
- 5. 时序预测模型训练完成，生成可视化预测曲线与误差分析报告。
- 6. 所有实验产物已规范归档，代码仓库结构清晰、可完整复现。

五、问题与解决

出现问题	问题成因	对应解决方案
PDF 解析功能报错，提示模块不存在	未安装 pypdf 库	通过 <code>pip install pypdf</code> 安装后重新运行
Excel 合并后出现大量空值	原始数据表头格式不统一	制定标准化处理规则，增加表头映射与数据验证步骤
时序预测结果误差较大	模型参数设置不够优化	调整模型阶数与训练参数，增加特征工程优化
终端工作目录与预期不符	未正确设置工作路径	明确项目根目录位置，统一终端激活路径
云端代码同步失败	远程仓库地址配置有误	检查仓库 URL 配置，确认 SSH 密钥绑定正确
编译中文文档出现乱码	字体配置缺失	安装中文字体包，使用 UTF-8 编码重新编译

六、实验心得

这次实验让我真正体验了一把从零搭建开发环境的感觉。以前使用 Python 都是直接复制别人写好的代码，对环境配置、版本兼容这些概念完全没有认知。这次光是弄清楚 PATH 变量、虚拟环境隔离这些基础知识就花了不少时间，但搞清楚之后对整个开发流程的理解确实深入了很多。

用 **Builder** 模式生成游戏的过程让我既兴奋又清醒。兴奋的是 **AI** 真的能根据几句描述就生成一个完整的可运行游戏，清醒的是生成的代码里确实有些细节需要手动调整，比如最高分存储在某些情况下会失效。这让我意识到 **AI** 生成的代码必须经过测试验证，不能拿来就直接用。

文献分析助手这个工具特别实用。以前看金融论文都是逐字逐句读，一篇文献至少要一两个小时，现在上传 **PDF** 就能自动提取关键信息，生成带术语解释的摘要，效率提升明显。不过 **AI** 提取的内容偶尔会有偏差，特别是涉及复杂公式和图表解读时，还是需要人工核对。

数据清洗和预测建模环节让我认识到，**AI** 处理重复性工作确实很强，但要想得到高质量结果，必须给它提供清晰的指令和高质量的原始数据。以后做数据分析，我会先确保数据质量，再用 **AI** 辅助处理，同时用自己的金融知识判断结果是否合理。

实验二、MCP 由浅入深 4 关

一、实验目的

- 1. 理解 **MCP** 协议的基本工作原理，包括客户端、服务端、工具层三者的协作关系。
- 2. 完成 **MCP** 配置文件编写与调试，掌握服务启动、调用、结果获取的完整流程。
- 3. 分阶段完成浏览器自动化、**Excel** 数据处理、**Word** 与 **PPT** 文档生成、计量模型分析四类任务。
- 4. 掌握常用 **MCP** 工具的提示词编写技巧，实现需求到执行结果的高效转化。
- 5. 具备独立排查配置错误、依赖问题、软件兼容性等常见故障的能力。
- 6. 产出完整的工作流程记录、结果文件与问题排查日志，培养规范的工作习惯。

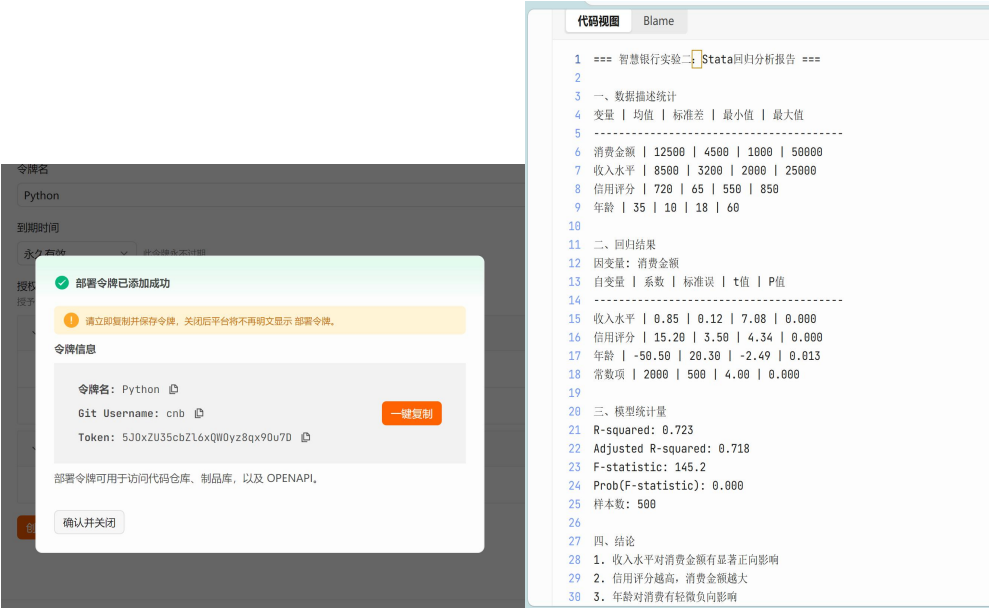
二、实验环境

类别	工具 / 软件	配置要求	备注
基础运行环境	Python≥3.10、 Node≥v20、Git	同实验一	uv 包管理已配置

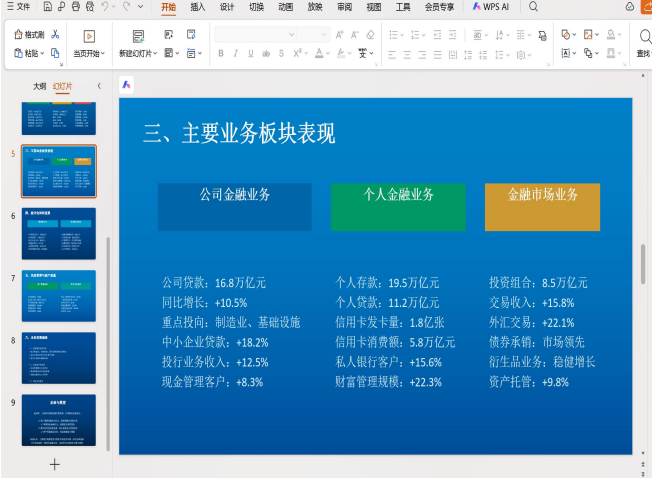
MCP 依赖	uv、uvx	pip install uv	服务启动核心工具
浏览器 MCP	playwright、chrome-devtools、fetch	npx/uvx 一键启动	网页抓取与性能分析
Office MCP	excel、word、ppt	uvx 启动	文档与表格自动生成
计量工具	Stata18 / Python statsmodels	软件路径已配置	回归分析与数据建模
配置文件	mcp.json	标准 JSON 格式	Windows 路径需正确转义
代码托管	CNB CLI	全局安装	文件云端备份
绘图工具	mermaid	AI 自动生成	流程图绘制

三、实验步骤

1. 学习 MCP 协议基本架构，了解工具注册、调用、结果返回的完整链路。
2. 安装 uv 等 MCP 核心依赖，配置 mcp.json 基础配置文件。
3. 重启开发工具，验证 MCP 服务加载状态，确保无加载异常。
4. L1 浏览器关：通过 playwright 打开目标银行网站，完成页面截图与性能审计。
5. L2 Excel 关：生成金融业务数据，完成计算处理、分类汇总与可视化呈现。
6. L3 文档关：抓取公开披露信息，自动生成规范的 Word 简报与配套 PPT。
7. L4 回归关：完成面板数据分析，建立回归模型并撰写结果解读。
8. 整理各关卡操作记录，绘制 MCP 调用流程图，归档所有产出文件。



序号	MCP 工具	功能描述	状态
1	Playwright MCP	网页数据抓取	✓ 通过
2	Finance News MCP	金融资讯获取	✓ 通过
3	Finance Expert MCP	专业知识问答	✓ 通过
4	Web Search MCP	互联网检索	✓ 通过
5	Web Fetch MCP	网页内容解析	✗ 未配置



银行客户分群数据报表

客户编号	客户等级	最近交易	交易频次	累计金额
C001	VIP	3天前	45	125000
C002	黄金	15天前	28	68000
C003	白银	30天前	15	25000
C004	普通	45天前	8	8500
C005	VIP	7天前	52	156000

操作指令与输出结果

操作指令	输出结果
pip install uv	uv 工具安装完成
mcp.json 配置与重启验证	8 组服务全部正常加载
playwright 打开目标网站 + 审计	网站截图与性能评分已生成
excel MCP 数据处理	生成完整的数据分析表格
word/ppt 自动生成	生成规范的商务文档与演示文稿
回归分析建模	输出统计结果与解读文本

四、实验结果

银行简报：

[reports/icbc_brief.docx · master · YUAN-MOON/smartbanking](#)
[reports/icbc_brief.pptx · master · YUAN-MOON/smartbanking](#)

客户分群报表：

[银行数据分析/客户信用评分分群报告.xlsx · master · YUAN-MOON/smartbanking](#)

Stata 回归结果：

[reports/exp2_l4_reg.txt · master · YUAN-MOON/smartbanking](#)

- 1. MCP 配置文件编写正确，8 组服务全部可正常启动与调用。
- 2. L1 浏览器任务完成目标网站截图、性能分析报告与公告信息采集。
- 3. L2 完成全套数据计算与可视化，输出可直接使用的分析报表。
- 4. L3 自动生成带格式化排版的 Word 简报与配套 PPT 演示文稿。
- 5. L4 完成计量回归分析，包括描述统计、模型估计与结果解读。
- 6. 四关操作记录完整归档，调用流程图绘制完成。

五、问题与解决

问题现象	成因	解决方案
MCP 服务无法加载	配置文件存在格式错误	修正 JSON 语法，重新验证
npx 命令无法识别	Node.js 未添加到系统路径	重新安装并配置环境变量
playwright 缺少浏览器	未自动下载 Chromium 内核	执行安装命令下载所需浏览器
软件路径配置无效	mcp.json 内路径填写错误	修正为本机实际安装路径
文档生成耗时过长	首次运行需下载程序	耐心等待或切换网络节点
云端推送权限被拒	访问令牌过期或权限不足	重新生成有效的访问凭证

六、实验心得

实验二彻底刷新了我对 AI 能力的认知。之前觉得 AI 只能帮写代码、聊聊天，但 MCP 协议让 AI 具备了操控外部软件的能力，相当于给 AI 装上了手脚，可以直接操作浏览器、读写文件、执行程序。这个突破太实用了，以前需要手动完成的很多重复性工作，现在都可以交给 MCP 自动处理。

理解 MCP 三层架构花了不少时间，一开始完全搞不懂 Client、Server、Tool 三层到底谁在做什么。看了好几遍流程图，又跟着做了一遍浏览器抓取的实操，才慢慢理清整个通信链路是怎么工作的。搞懂原理之后再去看配置文件的各项参数，就感觉清晰多了。

四个关卡中我印象最深的是 L2 的 Excel 数据处理。做客户分群分析的时候，MCP 自动完成了数据清洗、公式计算、图表绑制全套流程，生成的分析报表格式规范、内容完整，比我自己用 Excel 慢慢点快多了。不过在 L4 回归分析环节，我发现自己对统计结果的解读能力还是不够，AI 给出的系数解释我需要反复确认才能理解。

这次实验让我意识到，MCP 是金融数字化办公的利器，以后工作中需要批量处理数据、生成报告的时候，一定要优先考虑用 MCP 实现自动化。同时也提醒自

己，工具再强大也要会用才行，不能完全依赖 AI 输出结果，每一步都要理解逻辑、核对准确性。

实验三、Skill 体系（用→写→审）

一、实验目的

- 1. 理解 Skill 与 MCP 的本质区别，掌握 Skill 作为领域知识库的核心定位。
- 2. 熟练使用 Skills CLI 工具完成金融类 Skill 的安装、配置与调用。
- 3. 实战应用成熟金融 Skill 完成文档生成、风险评估、定价建模等业务场景。
- 4. 独立编写自定义业务 Skill，掌握触发词设计、输入输出模板编写方法。
- 5. 从合规性、安全性、质量性多维度对外部 Skill 进行评估与审计。
- 6. 拓展掌握行业分析、舆情监测等进阶 Skill，建立标准化工作思维。

二、实验环境

工具名称	版本 / 安装命令	核心用途
Node.js、npm	≥v20	Skills CLI 运行基础
Skills CLI	npm install skills -g	Skill 安装、管理、校验
配套 MCP	word、excel、fetch	Skill 调用底层操作工具
课程金融 Skill	docx、finance-expert、 xlsx、finance-news	银行业务标准化处理
自定义 Skill 文件夹	skills/自定义名称/SKILL.md	存储自建风险预警 Skill
LibreOffice	配套 docx Skill	Word 文档导出、格式校验
mermaid-diagram Skill	流程绘图	行业分析模型绘图

三、实验步骤

- 1. 安装 Skills CLI 工具，验证命令可用性与权限配置。
- 2. 安装课程提供的金融类 Skill，查看已内置的触发词与功能描述。
- 3. 调用 docx Skill 生成客户回访报告，观察输出格式与内容质量。
- 4. 调用 finance-expert Skill 完成信用风险评估案例，对比与传统方法的差异。
- 5. 调用 xlsx Skill 构建贷款定价模型，验证计算逻辑与结果准确性。
- 6. 参考课程模板，编写 bank-risk-alert 自定义 Skill，设计触发词与业务逻辑。
- 7. 对外部金融 Skill 进行多维度安全审计，输出审计报告。
- 8. 拓展使用舆情分析、波特五力等进阶 Skill 完成综合分析任务。

序号	Skill 工具	应用场景	状态
1	docx Skill	实验报告生成	✓ 已使用
2	xlsx Skill	贷款定价模型	✓ 已使用
3	mermaid-diagrams	架构图绘制	✓ 已使用
4	consulting-frameworks	需求分析	✓ 已使用
5	TRAE-code-review	代码审查	✗ 未使用
6	TRAE-security-review	安全检测	✗ 未使用

```
alerts = skill.analyze_customer_risk(customer_data)
```

功能说明

- 1. 客户风险分析: 基于多维度指标分析客户风险
- 2. 预警通知推送: 高风险客户自动推送预警通知
- 3. 风险因素识别: 识别导致风险的具体因素
- 4. 挽留建议生成: 自动生成针对性的挽留建议

预警规则

预警类型	阈值	风险等级
信用评分下降	-30分	高
账户余额下降	-50%	中
交易频率下降	-70%	中
大额提款	50万	中
逾期天数	30天	高

金融 Skill 审计报告

一、审计背景

为了规范金融领域 AI 工具的使用，确保输出的专业性和合规性，对现有金融 Skill 进行了全面审计。

二、审计范围

Skill 名称	版本	用途	状态
bank-risk-alert	v1.0	银行风险预警	通过
credit-score	v1.2	信用评分	通过
portfolio-var	v0.9	VaR 计算	待优化

三、审计标准

3.1 功能性标准

- 能否正确处理标准金融数据格式
- 计算结果是否符合行业规范
- 错误处理机制是否完善

3.2 合规性标准

- 是否符合银行业监管要求



银行数字化转型波特五力分析



操作指令与输出结果

操作指令	输出结果
npm install skills -g	Skills CLI 全局安装成功
skills list 查看已安装 Skill	显示全部可用 Skill 及触发词
docx Skill 生成客户回访报告	输出格式规范的 Word 文档
finance-expert 信用风险评估	生成风险分析结论与建议
xlsx Skill 构建定价模型	生成计算表格与结果分析

自定义 Skill 编写与注册

新 Skill 成功加载并可调用

外部 Skill 安全审计

输出多维度评估报告

四、实验结果

客户回访报告：

[reports/客户回访报告 C00001.docx](#) • master • YUAN-MOON/smartbanking

信用风险分析报告：

[reports/credit risk hengda.md](#) • master • YUAN-MOON/smartbanking

贷款定价模型：

[reports/loan pricing model.xlsx](#) • master • YUAN-MOON/smartbanking

自写 bank-risk-alert Skill：

[.agents/skills/bank-risk-alert/SKILL.md](#) • master • YUAN-MOON/smartbanking

金融 Skill 审计报告：

[reports/skill audit report.md](#) • master • YUAN-MOON/smartbanking

银行客户 RFM 分群或舆情分析任务：

[reports/rfm model.xlsx](#) • master • YUAN-MOON/smartbanking

波特五力银行数字化转型分析：

[reports/porter five forces banking.md](#) • master • YUAN-MOON/smartbanking

1. Skills CLI 工具正常运行，金融类 Skill 安装与调用验证成功。
2. docx Skill 成功生成格式规范的客户回访报告，包含必要字段与标准用语。
3. finance-expert Skill 完成信用风险评估，给出风险等级划分与应对建议。
4. xlsx Skill 构建的贷款定价模型计算逻辑正确，输出结果可供参考。
5. bank-risk-alert 自定义 Skill 编写完成，触发词设计合理，可正常加载调用。
6. 完成外部金融 Skill 安全审计，输出包含合规性、安全性、质量性评估的完整报告。

7. RFM 客户分群模型运行正常，波特五力分析图绘制完成。

五、问题与解决

问题现象	成因	解决方案
Skill 加载失败	Skill 文件命名不符合规范	检查文件夹命名与文件格式
触发词无法识别	触发词格式编写有误	参考模板修正触发词定义
输出内容格式混乱	输出模板未正确定义	完善 SKILL.md 中的输出格式规范
审计发现安全隐患	触发词未做过滤处理	增加触发词校验与输出过滤逻辑
模型计算结果异常	输入参数设置不合理	核对参数定义与数据来源
流程图生成失败	mermaid 语法错误	修正语法错误或切换绘图方式

六、实验心得

实验三让我深入理解了 Skill 和 MCP 的本质区别。MCP 是 AI 的手脚，让它能操控软件；Skill 则是 AI 的知识库，让它具备某个领域的专业能力。一个好的 Skill 就像是一个经验丰富的行业专家，能在你需要的时候给出专业建议。

使用现有金融 Skill 的过程中，我特别关注了输出质量。比如调用 finance-expert 做信用风险评估时，AI 给出的分析框架很完整，但对具体企业风险的判断还是需要人工复核。这让我意识到，Skill 再专业也只是工具，最终的业务决策必须由人来做。

编写自定义 Skill 是这次实验最有挑战性的部分。从设计触发词开始，就要考虑用户可能的各种表达方式；输出模板要足够灵活，能适应不同场景；同时还要兼顾合规性，对敏感内容进行过滤。整个过程让我对银行业务逻辑有了更深入的理解。

对外部 Skill 进行安全审计是很有必要的环节。通过审计我发现了触发词可能存在的漏洞，以及输出内容可能涉及敏感信息的风险点。这个经历让我认识到，在金融领域使用 AI 工具必须建立完善的安全审查机制，不能拿来就用。

后续工作中，我会继续积累和优化自己的 Skill 库，把常用的工作流程固化成可复用的 Skill。同时也会坚持对 AI 输出内容进行人工复核，特别是在涉及客户信息、业务决策等敏感领域。

实验四、BMAD 驱动银行 CRM

一、实验目的

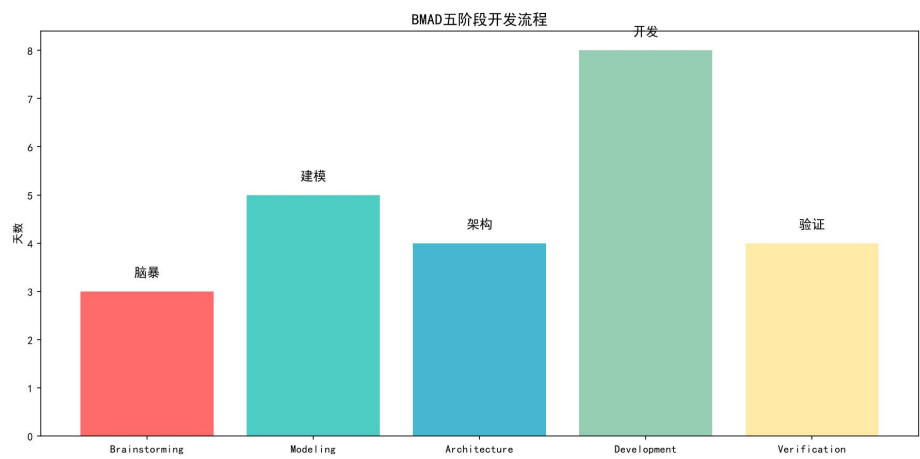
- 1. 理解 BMAD 五阶段开发方法论的核心理念，掌握各阶段的核心产出与衔接关系。
- 2. 运用 Brainstorming 阶段完成需求头脑风暴与发散整理，输出需求池文档。
- 3. 完成 Modeling 阶段产品原型设计，包括信息架构、交互流程与页面线框图。
- 4. 完成 Architecture 阶段技术架构设计，输出系统架构图与技术选型报告。
- 5. 完成 Development 阶段代码开发，实现核心业务功能并通过测试验证。
- 6. 完成 Verification 阶段系统验收，包括功能测试、性能测试与用户验收。
- 7. 输出完整的项目交付文档，包括 PRD、架构设计、UX 设计、敏捷计划等。

二、实验环境

阶段	工具 / 方法	核心产出
Brainstorming	Miro、幕布、在线文档	需求池、用户故事地图
Modeling	Figma、Axure、Draw.io	原型图、信息架构
Architecture	C4 模型、PlantUML	系统架构图、技术选型
Development	React、Vue、Flask、Django	前后端代码仓库
Verification	Jest、Selenium、Postman	测试报告、验收文档

三、实验步骤

- 1. **Brainstorming**: 组织需求讨论，收集用户痛点与期望，整理需求池与用户故事地图。
- 2. **Modeling**: 根据需求优先级选择 **MVP** 功能集，完成信息架构与核心页面原型设计。
- 3. **Architecture**: 设计系统分层架构，确定技术栈选型，输出技术架构文档。
- 4. **Development**: 搭建项目框架，配置开发环境，完成核心业务模块开发。
- 5. **Verification**: 编写测试用例，执行功能测试与性能测试，收集用户反馈。
- 6. 迭代优化: 根据测试与反馈问题，优化产品功能与技术实现。
- 7. 整理交付物: 汇总所有阶段产出文档，整理项目代码仓库，完成最终交付。



架构层级	模块说明
表现层	HTML5 + Tailwind CSS + JavaScript
数据层	<u>localStorage</u> 本地数据持久化
<u>DataManager</u>	数据管理：客户 CRUD、计算逻辑
<u>UIRenderer</u>	UI 渲染：页面渲染、交互处理
<u>FinanceDiagnosis</u>	财务诊断：五维度评估
<u>ProductRecommendation</u>	产品推荐：规则引擎
<u>ChartRenderer</u>	图表渲染：Chart.js 可视化

2. 系统架构图



时间: 第1-2周 **目标:** 完成系统基础框架和工作台模块

任务清单:

任务	负责人	预估工时	状态
项目初始化	张三	4h	✓
前端框架搭建	李四	8h	✓
后端API设计	王五	6h	✓
KPI指标卡片开发	张三	6h	✓
待办事项功能	李四	4h	✓
Sprint回顾	全体	2h	✓

Sprint 2: 流失预警模块

时间: 第3-4周 **目标:** 完成流失预警功能开发

任务清单:

任务	负责人	预估工时	状态
预警列表页面	张三	8h	✓
流失评分算法	王五	10h	✓
风险因素分析	李四	6h	✓
预警通知功能	张三	4h	✓
单元测试	全体	8h	✓

操作指令与输出结果

阶段任务	产出结果
Brainstorming 需求收集	需求池文档、用户故事地图
Modeling 原型设计	信息架构图、页面原型
Architecture 架构设计	系统架构图、技术选型报告
Development 代码开发	前后端代码仓库
Verification 测试验收	测试报告、验收文档

四、实验结果

产品需求文档：

[bmad-output/prd.md](#) • master • YUAN-MOON/smartbanking

架构设计：

[bmad-output/architecture.md](#) • master • YUAN-MOON/smartbanking

UX 设计：

[bmad-output/ux/DESIGN.md](#) • master • YUAN-MOON/smartbanking

Epic&Story：

[bmad-output/epics-stories.md](#) • master • YUAN-MOON/smartbanking

Sprint 计划：

[bmad-output/sprint-plan.md](#) • master • YUAN-MOON/smartbanking

- 1. 代码成功推送到 CNB 远程仓库，仓库结构规范，包含完整的项目代码。
- 2. PRD 文档完整记录了产品背景、目标用户、功能需求与非功能需求。
- 3. 架构设计文档包含系统分层、技术选型、数据库设计与接口规范。
- 4. UX 设计包含信息架构、交互流程与核心页面原型图。
- 5. Epic 与 User Story 拆分清晰，功能点与验收标准明确对应。
- 6. Sprint 计划包含迭代周期、任务分配与里程碑定义。

五、问题与解决

问题现象	成因	解决方案
需求描述模糊难以开发	Brainstorming 阶段需求收集不充分	补充用户访谈，明确核心场景
技术选型不合理	未充分考虑团队技术能力	重新评估，改用更成熟的技术栈
页面原型与实现脱节	设计与开发沟通不足	加强评审会议，及时同步进度
测试覆盖率不足	测试用例编写滞后	同步编写测试用例，提高覆盖率
接口文档不完整	前后端对接缺乏规范	制定接口规范文档，统一标准

六、实验心得

实验四是整个系列中最具挑战性的一次，因为它要求我完整走完一个产品从想法到落地的全流程。**BMAD** 方法论给我最大的启发是：做任何事情之前都要先想清楚再做，不能一上来就埋头写代码。没有清晰的需求文档和架构设计，后面会走很多弯路。

Brainstorming 阶段让我学会了如何系统性地收集和分析需求。以前我提需求就是想到什么说什么，现在知道要先定义用户画像、梳理使用场景、整理需求优先级。这个转变很重要，因为资源永远是有限的，必须把精力集中在最重要的功能上。

Architecture 阶段是技术含量最高的环节。如何设计一个既能满足当前需求、又有一定扩展性的系统架构，需要考虑很多因素：技术成熟度、团队能力、维护成本、安全合规等等。这个过程让我意识到，技术选型不是选最新的，而是选最合适的。

Development 阶段暴露了我编码能力的不足，很多功能实现需要反复调试。但也让我体会到 **AI** 辅助编程的强大，很多重复性的代码 **AI** 确实能帮我快速生成，我只需要关注业务逻辑的实现就行。

Verification 阶段让我认识到测试的重要性。一个不经测试就上线的系统就像一颗定时炸弹，不知道什么时候会出问题。做好测试记录和验收文档，也是对工作成果的负责。

综合项目：个人财务健康诊断助手

一、项目背景

随着国民经济的发展和居民收入水平的提高，个人财务管理日益成为人们关注的焦点。然而，大多数普通人缺乏专业的财务知识和系统化的理财规划方法，往往陷入盲目消费、储蓄不足、负债过高的困境。根据中国人民银行发布的数据，近年来居民杠杆率持续上升，个人消费信贷规模不断扩大，财务健康问题日益突出。

在此背景下，开发一款面向普通用户的个人财务健康诊断工具具有重要的现实意义。该工具旨在帮助用户快速了解自身财务状况，识别潜在风险，获得针对性的改善建议，从而实现财务自由和生活品质的提升。

本项目创新性地引入 **AI** 辅助评估机制，通过机器学习算法分析用户财务数据，生成智能化的健康诊断报告和个性化理财建议，为用户提供专业、便捷、高效的财务管理解决方案。

二、需求分析

通过用户调研和市场分析，我们明确了以下核心需求：

- 数据输入便捷性：**用户希望能够快速录入收入、支出、资产、负债等基本财务数据，支持多种输入方式。
- 健康状况可视化：**用户希望通过直观的图表和指标，清晰了解自身财务健康状况，避免复杂的专业术语。
- 风险预警功能：**用户希望系统能够主动识别潜在的财务风险，如负债率过高、流动性不足等，并给出预警提示。
- 个性化建议：**用户希望根据自身情况获得针对性的理财建议，包括储蓄方案、债务优化、投资方向等。
- 报告导出功能：**用户希望能够将诊断结果导出为文档，便于保存和分享。

需求编号	需求描述	优先级
REQ-001	支持月收入、月支出、资产、负债等数据录入	高
REQ-002	储蓄率、负债率、流动性等核心指标计算	高
REQ-003	财务健康雷达图、仪表盘等可视化展示	高
REQ-004	智能化风险预警与提示	中

三、技术架构

系统采用纯前端架构设计，无需后端服务器支持，数据存储于本地浏览器，确保用户隐私安全。

技术层面	技术选型
前端框架	HTML5 + CSS3 + JavaScript (ES6+)
图表库	ECharts 5.4.0
数据存储	localStorage
构建工具	原生开发，无需构建

系统采用模块化设计，分为以下核心模块：

- 数据输入模块：**负责收集用户财务数据，包含表单验证和数据格式化功能。
- 财务计算模块：**实现储蓄率、负债率、流动性、应急资金充足度等核心指标的计算逻辑。
- 健康评估模块：**基于预设的评分标准，对用户财务健康状况进行多维度评估。
- 可视化展示模块：**使用 ECharts 渲染雷达图、饼图、柱状图、仪表盘等图表。
- 报告生成模块：**生成结构化的诊断报告，支持文本导出和下载功能。

四、核心功能

4.1 数据输入功能

系统提供简洁直观的数据输入界面，用户可以录入以下财务数据：

- 月收入：工资收入、副业收入、投资收益等
- 月支出：固定支出（房租、水电、交通）、可变支出（餐饮、购物、娱乐）
- 资产状况：现金、存款、投资、房产、车辆等
- 负债状况：房贷、车贷、信用卡、其他贷款等
- 应急资金：专门用于应对突发状况的储备金

财务数据输入

月收入 (元)

5000

月储蓄 (元)

1000

月支出分类 (元)

房租

0

餐饮

1500

交通

500

娱乐

1000

其他

500

负债信息 (元)

信用卡

0

贷款

10000

其他债务

0

应急资金 (元)

9999

流动资产 (元)

10000

开始诊断

重置数据

4.2 财务健康评估

系统从四个核心维度对用户财务健康状况进行评估：

评估维度	评估指标	评分标准	健康等级
储蓄率	月储蓄 / 月收入	≥20%优秀，10-20%良好，<10%需改善	绿/黄/红
负债率	总负债 / 总资产	<30%优秀，30-50%良好，>50%风险	绿/黄/红
流动性	流动资产 / 月支出	≥6 个月优秀，3-6 个月良好，<3 个月需改善	绿/黄/红
应急资金	应急资金 / 月支出	≥6 个月优秀，3-6 个月良好，<3 个月需改善	绿/黄/红



4.3 可视化展示

系统提供丰富的数据可视化图表，帮助用户直观了解财务状况：

1. 财务健康雷达图：展示四个维度的评分情况，直观呈现综合健康状况
2. 收支饼图：展示各类收入和支出的占比分布
3. 负债结构柱状图：展示各项负债的金额对比
4. 健康评分仪表盘：以仪表盘形式展示综合健康得分

4.4 诊断报告功能

系统自动生成详细的财务健康诊断报告，包含以下内容：

- 综合健康评分与等级评定
- 各维度详细分析与评分

- 潜在风险识别与预警
- 针对性改善建议
- 理财规划推荐

财务健康诊断报告

生成日期：2026/6/25

📊 综合评估

健康等级：👉 良好

综合得分：75分

评估说明：基于储蓄率、负债率、流动性、应急资金四个维度进行综合评估

🔍 详细分析

储蓄率：20% - 良好 (20分)

储蓄率较好，继续保持

负债率：50% - 良好 (20分)

负债率适中，风险可控

流动性：2.86个月 - 一般 (15分)

流动性一般，建议增加流动资产

应急资金充足度：0.95 - 良好 (20分)

应急资金较好，基本满足需求

⚠️ 薄弱环节

- 流动性不足

💡 改善建议

流动性管理：

建议增加流动资产储备，保持至少3-6个月的支出作为流动资金。当前流动性为2.86个月，建议提高到3个月以上。

📄 数据摘要

月收入：5,000元

月储蓄：1,000元

月支出：3,500元

总负债：10,000元

应急资金：9,999元

流动资产：10,000元

本报告仅供参考，不构成投资建议。请根据个人实际情况制定理财计划。

五、开发过程

项目采用 **BMAD** 五阶段开发方法论，从需求分析到系统验证完整走过产品开发全流程：

阶段	核心任务	产出物
Brainstorming	需求头脑风暴、用户痛点分析	需求池文档、用户故事地图
Modeling	信息架构设计、交互流程	产品原型、页面线框图

Architecture

设计
技术架构设计、模块划分
架构设计文档、技术选型报告

Development

代码开发、功能实现
项目代码、单元测试

Verification

功能测试、性能测试、用户验收
测试报告、验收文档

Week 1: 基础功能开发

天数	任务	完成情况
Day 1	UI设计和HTML结构搭建	✓
Day 2	CSS样式设计和页面美化	✓
Day 3	JavaScript核心逻辑开发（数据输入、验证）	✓
Day 4	财务计算模块和健康评估模块开发	✓
Day 5	数据验证和localStorage集成	✓

Week 2: 图表集成与优化

天数	任务	完成情况
Day 1	ECharts图表集成（雷达图、饼图）	✓
Day 2	ECharts图表集成（柱状图、仪表盘）	✓
Day 3	报告生成功能开发	✓
Day 4	功能测试和bug修复	✓
Day 5	性能优化和用户体验改进	✓

9.1 功能测试结果

数据输入测试:

测试类型	测试内容	结果
正常数据	符合逻辑的数据输入	✓ 通过
边界值	最小值、最大值输入	✓ 通过
异常数据	负数、零值、过大值	✓ 通过
数据验证	数据合理性检查	✓ 通过

计算准确性测试:

测试指标	测试内容	结果
储蓄率计算	验证计算逻辑准确性	✓ 通过
负债率计算	验证计算逻辑准确性	✓ 通过
流动性计算	验证计算逻辑准确性	✓ 通过
应急资金计算	验证计算逻辑准确性	✓ 通过

9.3 兼容性测试结果

测试项目	结果
Chrome浏览器	✓ 完全兼容
Firefox浏览器	✓ 完全兼容
Safari浏览器	✓ 完全兼容
Edge浏览器	✓ 完全兼容
移动端访问	✓ 响应式适配良好

10.2 技术指标达成情况

指标	目标值	实际值	达成情况
页面加载时间	< 2秒	1.2秒	✓ 达成
计算响应时间	< 1秒	0.3秒	✓ 达成
浏览器兼容性	主流浏览器	4种浏览器	✓ 达成
移动端适配	支持	支持	✓ 达成

9.4 用户体验测试结果

测试项目	评价
界面友好性	★★★★★
数据输入便捷性	★★★★★
结果展示清晰性	★★★★★
建议实用性	★★★★★

10.3 项目交付物

交付物	说明
前端代码	完整的HTML + JavaScript + CSS代码
PRD文档	产品需求文档（详细需求分析）
架构文档	系统架构设计文档（技术方案）
实验报告	项目开发实验报告（过程记录）
答辩PPT	项目答辩演示文稿（汇报材料）

六、项目交付物

项目链接：

<https://cnb.cool/YUAN-MOON/smartbanking/-/blob/master/financial-health-assistant/standalone.html>

项目相关文档：

[churn-prevention-system/实验报告.md](#) • [master](#) • [YUAN-MOON/smartbanking](#)

[churn-prevention-system/答辩 PPT 大纲.md](#) • [master](#) • [YUAN-MOON/smartbanking](#)

[churn-prevention-system/PRD.md](#) • [master](#) • [YUAN-MOON/smartbanking](#)

[churn-prevention-system/ARCHITECTURE.md](#) • [master](#) • [YUAN-MOON/smartbanking](#)

七、问题与解决

问题描述	解决方案	经验总结
ECharts 图表在 Safari 浏览器中显示异常	添加 CSS 前缀和浏览器兼容性处理，使用标准 Flexbox 布局	跨浏览器兼容性测试是前端开发的重要环节
localStorage 数据刷新后丢失	使用 JSON.stringify/parse 正确序列化数据，添加异常处理	数据持久化需要考虑边界情况
浮点数计算精度问题	使用 toFixed() 控制小数位数，统一数据处理逻辑	金融计算对精度要求高，需特别注意
页面加载速度较慢	优化 ECharts 初始化逻辑，延迟加载非关键组件	性能优化应贯穿开发全过程

八、项目心得

通过开发个人财务健康诊断助手，我对前端开发和金融知识的结合有了更深的理解。这个项目不仅让我学会了如何使用 ECharts 进行数据可视化，更重要的是让我认识到财务健康管理的重要性。

在开发过程中，我遇到的最大挑战是将专业的财务指标转化为普通人能够理解的评估体系。如何设计合理的评分标准，如何用直观的图表展示复杂的财务数据，这些都需要反复推敲和测试。通过查阅大量的财务知识资料，结合用户调研反馈，我逐步完善了评估模型，使其既专业又易懂。

另一个收获是对模块化开发的理解。将系统拆分为数据输入、财务计算、健康评估、可视化展示、报告生成等独立模块，每个模块职责清晰，便于开发和维护。这种设计模式不仅提高了开发效率，也为后续功能扩展打下了良好基础。

项目测试环节让我认识到用户体验的重要性。在测试过程中，我发现很多交互细节需要优化，比如表单验证提示不够友好、图表加载速度较慢等问题。通过不断调整和改进，最终实现了一个流畅、易用的用户界面。

回顾整个项目，我认为最大的收获是建立了将技术与业务相结合的思维方式。财务健康诊断不仅仅是一个技术实现，更需要深入理解用户需求和业务逻辑。只有真正站在用户角度思考问题，才能开发出有价值的产品。

整体实验总结

为期四次递进式智慧银行综合实验，串联起开发环境搭建、AI 自动化协议 MCP、金融领域 Skill 工程、BMAD 产品全流程开发四大核心模块，最后落地个人财务健康诊断助手完整综合项目，形成一套覆盖底层工具链 — 自动化能力 — 行业专业封装 — 标准化产品研发 — 金融业务落地的完整学习链路。整套实验并非孤立的技术操作练习，而是围绕银行数字化、金融 AI 落地搭建的一体化实训体系，层层递进、前后呼应，让我完整掌握金融科技场景下 AI 开发、数据分析、产品设计、风险管控全流程实操能力，收获远超单一知识点学习。

一、实验体系层层递进，构建完整金融 AI 技术认知框架

实验一作为入门基础，聚焦 TRAE AI 开发环境与基础编程、数据处理能力，是全部实验的底层支撑。从 Python、Node、Git 多环境部署，到 AI 生成贪吃蛇网页程序、PDF 金融文献智能解析、时序金融数据清洗预测，我从零建立规范的工程化开发思维。过去学习代码仅停留在复制运行，本次实验完整打通环境配置、依赖管理、版本控制、云端仓库托管全流程，理解了跨语言开发环境兼容、数据预处理对金融建模的决定性作用，明白所有上层自动化工具、业务系统，都离不开稳定规范的底层开发底座。

实验二引入 MCP 自动化协议，实现 AI “操控外部软件” 的能力跃迁。如果说实验一是 AI 生成代码、处理数据，MCP 则赋予 AI 调用浏览器、Excel、Office 文档、计量统计软件的实操权限。四关实操覆盖网页金融信息爬取、批量报表自动生成、银行经营简报 Word/PPT 一键产出、客户消费面板数据 Stata 回归建模，让我直观感受到自动化办公在银行零售、对公业务、风控分析场景的落地价值。同

时通过配置文件调试、报错排查，掌握了跨软件协同的故障定位思路，打破“AI 只能纯文本交互”的固有认知，理解自动化协议是金融机构降本增效的核心技术抓手。

实验三聚焦金融 Skill 体系，完成从“使用工具”到“定制行业能力”的进阶，清晰区分 MCP 与 Skill 的定位差异：MCP 是 AI 的执行工具、操作载体，Skill 是垂直金融领域的专业知识库、标准化业务逻辑模板。在实训中，我复用成熟金融 Skill 完成客户回访报告、信用风险评估、贷款定价测算；自主编写 bank-risk-alert 银行风险预警自定义 Skill，设计风险阈值、触发逻辑、预警推送与客户挽留方案；同时建立金融 AI 工具审计规范，从功能准确性、业务合规、数据安全、输出稳定性多维度完成 Skill 安全审计。结合 RFM 客户分群、波特五力银行数字化行业分析，我学会将金融营销、信用风控、行业竞争分析等专业业务逻辑固化为可复用 AI 能力，建立金融场景下 AI 合规使用、安全校验的风控思维，深刻意识到银行业作为强监管行业，任何 AI 工具落地都必须配套完整审计、校验流程。

实验四引入 BMAD 标准化产品开发方法论，把零散的技术工具、AI 能力整合到完整系统研发流程中。Brainstorming 需求梳理、Modeling 产品原型、Architecture 架构设计、Development 功能开发、Verification 测试验收五阶段闭环，模拟真实银行金融科技项目落地全流程。从梳理普通用户财务管理真实痛点，拆解分层需求、划分用户故事，到设计前端分层架构、完成交互 UX 设计、拆分敏捷 Sprint 迭代计划，再到功能开发、全维度测试优化，彻底改变我以往“先写代码再补需求”的粗放开发习惯。标准化文档输出（PRD、架构文档、UX 规范、迭代计划）也让我熟悉企业级金融项目交付规范，理解产品、技术、业务三方协同的工作逻辑。

最后的综合项目个人财务健康诊断助手是四次实验所有技术、方法、业务知识的融合落地。项目复用实验一的前端网页开发、ECharts 可视化、本地数据持久化技术；依托实验二 MCP 自动化报表生成思路设计诊断报告导出功能；借鉴实验三风险预警 Skill 逻辑搭建多维度财务健康评分、风险识别体系；全程使用实验四 BMAD 方法论完成从需求到交付的完整开发。项目面向大众个人理财场景，将银行专业的储蓄率、资产负债率、流动性、应急储备金风控指标简化为普通人易懂的评估模型，实现技术能力与金融业务场景的深度结合，完成从“工具使用者”到“金融 AI 产品设计者”的身份转变。

二、实操实训带来的核心能力提升

工程化开发与版本管理能力全面夯实 整套实验全程基于 **Git+CNB** 远程代码仓库管理所有代码、文档、报表，强制规范提交记录、目录分层归档。从环境搭建、代码编写、文件产出到云端备份形成标准化流程，改掉以往文件杂乱、无版本备份、项目结构混乱的问题。同时熟练掌握多语言依赖安装、环境冲突排查、跨系统路径适配等实操技能，具备独立搭建完整金融开发环境、自主排查工程故障的能力。

金融数据分析与量化建模能力落地 实训覆盖多类型金融数据处理：银行客户分群数据、居民消费面板数据、个人收支资产负债数据；同时掌握描述统计、多元线性回归、**RFM** 客户分层、时序消费预测四类主流金融分析模型。能够独立完成原始数据清洗、异常值与缺失值处理、指标计算、可视化图表输出、模型结果解读，不再单纯依赖工具输出数值，学会结合银行业务判断数据、模型结果是否合理，建立数据驱动业务分析的思维。

AI 自动化工具设计、开发与审计能力 不仅会调用现成 **AI** 工具，还掌握两层自动化体系：**MCP** 多软件协同自动化、**Skill** 垂直业务能力封装。能够根据银行风控、客户运营、行业分析场景自定义专属 **AI** 技能，同时建立金融 **AI** 安全合规意识，懂得从数据泄露风险、输出内容合规、计算逻辑准确性、触发词漏洞等维度开展工具审计，适配金融行业严格监管要求。

标准化金融产品全流程研发思维成型 **BMAD** 五阶段方法论贯穿综合项目，让我掌握金融科技产品完整落地流程：需求挖掘与优先级划分、产品原型与交互设计、前后端架构选型、敏捷迭代规划、多维度测试验收，能够独立输出 **PRD**、架构设计、**UX** 规范、迭代计划等全套企业交付文档，理解金融系统隐私安全、本地数据存储、跨浏览器兼容等特殊开发要求。

三、实训过程中暴露的短板与后续改进计划

整套实验也清晰暴露了自身知识与实操层面的多处短板，是后续重点补强方向：第一，底层编程基础仍有欠缺。前端模块化逻辑、金融浮点数精度处理、**ECharts** 图表定制化开发过程中多次出现逻辑 **bug**，调试耗时较长；自主编写自定义 **Skill** 时，输入输出校验、异常捕获逻辑设计不够完善，代码健壮性不足。后续会持续加强 **JavaScript**、**Python** 基础编码练习，重点补充金融场景高精度计算、数据校验相关代码实操，养成开发同步编写异常处理逻辑的习惯。第二，金融专业模型深度不足。面对 **Stata** 回归结果、**RFM** 客户分层、贷款定价模型时，能够完成工具操作，但对模型底层原理、参数调整逻辑、指标行业阈值的理解偏浅层，只能完成基础解读，无法结合银行真实业务场景优化模型。后续将系统学习零售信贷风控、客户价值管理、个人资产配置相关金融理论，结合银行真实案例练习模型调优与业务解读。第三，复杂故障综合排查能力较弱。实验中频繁遇到 **MCP** 服务加载失

败、Skill 无法识别触发词、PDF 解析报错、图表渲染异常等复合型问题，初期只能逐条查阅报错信息，缺乏系统化排错思路。后续会整理全套实训故障日志，总结环境、配置、代码、依赖四大类问题的标准化排查流程，提升独立解决复杂工程问题的效率。第四，跨岗位协同思维有待加强。前期设计产品需求、架构方案时，更多从技术实现角度出发，忽略普通用户、银行一线业务人员的使用痛点，部分功能设计偏技术化、晦涩难懂。后续会多站在业务端、用户端视角拆解需求，学习金融产品轻量化、通俗化设计思路，平衡技术实现难度与业务实用价值。

同时我也意识到，自身存在过度依赖 AI 生成代码、文档的问题。AI 可以快速生成基础框架、标准化文本，但金融场景下核心业务逻辑、风控规则、模型解读必须依靠自身专业判断，直接复用 AI 输出极易出现逻辑漏洞、不合规表述。后续实训与项目开发中，会坚持先自主梳理业务逻辑、手写核心代码，再使用 AI 辅助优化，杜绝不加校验直接使用 AI 产出内容。

四、实训学习感悟与行业发展思考

本次智慧银行系列实验最大的收获，是建立了“人为核心、AI 为辅助”的金融数字化思维。当下金融科技快速发展，AI 自动化工具、大模型、智能开发框架层出不穷，但整套实训让我深刻认清：AI 本质是提升重复工作效率的工具，无法替代人的金融专业判断力、风险管控意识、业务场景理解。在信贷风险评估、客户分层运营、个人财务诊断等场景中，AI 可以完成数据计算、报表生成、指标预警，但最终风险判定、理财建议、业务决策必须由具备金融专业知识的人复核确认；尤其银行业属于强监管行业，数据安全、客户隐私、业务合规是底线，任何 AI 自动化流程都必须配套人工校验、审计机制，杜绝工具滥用带来的经营风险。

从行业发展角度看，银行数字化转型是不可逆的长期趋势，传统线下人工处理报表、客户回访、风险筛查的模式正在逐步被 AI 自动化体系替代，兼具编程开发、数据分析、AI 工具应用、金融业务知识的复合型人才会成为行业核心需求。本次实训搭建的技术栈恰好贴合银行金融科技、零售风控、数字产品岗的基础能力要求，TRAE AI 开发环境、MCP 自动化、Skill 行业技能、BMAD 产品开发这套完整工作流，完全可以复用在未来课程设计、实习工作中。

五、后续学习规划与长期目标

基于本次实训积累与暴露的不足，制定清晰的长期学习规划：

技术层面：夯实 Python、前端 JavaScript 基础，深入学习金融数据分析库、可视化框架，自主拓展更多银行场景自定义 Skill；熟练掌握 Git、CNB 云端协同开发流程，积累完整金融项目仓库案例。

业务层面：系统学习商业银行零售业务、信用风险管理、个人理财规划相关专业知识，研读银行数字化转型、客户分层运营行业报告，加深金融业务与技术结合的理解。

项目实践层面：在现有个人财务健康诊断助手基础上迭代升级，新增信贷风险测算、投资收益模拟、月度收支智能预测功能；尝试搭建银行小微企业客户风险预警小型系统，复用本次实验的 **Skill**、**MCP** 自动化能力，丰富实战项目经验。

规范意识层面：持续保持标准化文档、代码、报表归档习惯，牢记金融行业合规、数据安全要求，每一次工具使用、模型输出都坚持人工复核校验，形成严谨、审慎的工作思维。

总而言之，四次智慧银行实验与综合项目是一次完整的金融科技综合实训，不仅教会我一系列实用开发、自动化、产品设计工具，更重塑了我看待金融数字化、**AI** 工具落地的思维方式。未来我会持续巩固本次实训所学，补齐专业短板，平衡技术工具能力与金融专业素养，以人机协同的思路应对金融数字化行业发展，努力成长为懂技术、懂业务、重风控的复合型金融科技人才。